

Raport z listy 8

Modele regresji i ich zastosowania

Tadeusz Jagniewski, album 277522

2026-05-06

Spis treści

1	Wstęp	1
2	Zadanie 1	2
3	Zadanie 2	2
4	Zadanie 3	3
5	Zadanie 4	5
6	Zadanie 5	5
7	Zadanie 6	6
8	Zadanie 7	7
9	Zadanie 8	8
10	Zadanie 9	8

1 Wstęp

Rozważamy model regresji nieliniowej

$$Y = g(\mathbf{x}, \beta) + \varepsilon.$$

Niech

$$n = 10, \quad (\beta_1, \beta_2, \beta_3, \sigma) = (10, 3, 2, 1/2) \quad x_i = \frac{i}{2} \quad \text{dla} \quad i = 1, 2, \dots, n.$$

2 Zadanie 1

Generujemy n obserwacji $y_1, \dots, y_n$ postaci

$$y_i = \frac{\beta_1 x_i}{\beta_2 + x_i} + \beta_3 + \varepsilon_i, \quad i = 1, 2, \dots, n,$$

gdzie $\varepsilon_1, \dots, \varepsilon_n$ są niezależnymi zmiennymi losowymi o rozkładzie $\mathcal{N}(0, \sigma^2)$.

```
set.seed(123)
n <- 10
x <- (1:n) / 2
beta.1 <- 10
beta.2 <- 3
beta.3 <- 2
sigma <- 1/2
sigma.kwadrat <- sigma^2
varepsilon <- rnorm(n, mean = 0, sd = sigma)
y <- (beta.1 * x) / (beta.2 + x) + beta.3 + varepsilon
```

3 Zadanie 2

Funkcja g jest postaci

$$g(x, \beta) = \frac{\beta_1 x}{\beta_2 + x} + \beta_3$$

Jej gradient G jest postaci

$$G(\mathbf{x}, \beta) = \nabla g(\mathbf{x}, \beta) = \begin{bmatrix} \frac{\partial g}{\partial \beta_1} \\ \frac{\partial g}{\partial \beta_2} \\ \frac{\partial g}{\partial \beta_3} \end{bmatrix} = \begin{bmatrix} \frac{x}{x+\beta_2} \\ \frac{-\beta_1 x}{(\beta_2+x)^2} \\ 1 \end{bmatrix}.$$

Implementujemy te wzory w funkcjach `g()` oraz `gradient()`.

```
g<-function(x,beta.1,beta.2,beta.3){
  return((beta.1 * x)/(beta.2+x) + beta.3)
}
```

```
gradient<-function(x,beta.1,beta.2,beta.3){
  g.1 <- x / (x+beta.2)
  g.2 <- ((-beta.1)*x) / (beta.2+x)^2
  g.3 <- 1
}
```

```

return(c(g.1,g.2,g.3))
}

```

4 Zadanie 3

Przyjmujemy następujące oznaczenia:

- $X = [x_{i,j}] \in \mathcal{M}(\mathbb{R}^{n \times p})$ – macierz wartości realizacji zmiennych objaśniających,
- $Y = [y_1, y_2, \dots, y_n]^T$ – wartości realizacji zmiennej objaśnianej,
- $\beta^{(0)} \in \mathbb{R}^p$ – początkowa wartość wektora β ,
- $g(X; \beta) = [g(x_1; \beta), g(x_2; \beta), \dots, g(x_n; \beta)]^T$ – wartości dla ustalonej funkcji g w oparciu o zadane parametry β i wartości realizacji zmiennych objaśniających,
- $G^{(j)} = \left[\frac{\partial g(x_i; \beta)}{\partial \beta_j} \right]_{i,j} \in \mathcal{M}(\mathbb{R}^{n \times p})$ – macierz gradientów funkcji g ,
- $L(\beta; \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log \sigma^2 - \sum_{i=1}^n \frac{(y_i - g(\cdot; \beta))^2}{2\sigma^2}$ – logarytm funkcji wiarygodności,
- $S(\beta) = \|Y - g(x; \beta)\|^2$ – kwadrat normy wektorów Y i $g(x; \beta)$.

Kolejne kroki algorytmu są następujące:

- wybieramy punkt startowy parametrów $\beta^{(0)} \in \mathbb{R}^p$ oraz liczby $\varepsilon > 0$ i $M \in \mathbb{N}_+$,
- tworzymy ciąg iteracji:

$$\beta^{(j+1)} = \beta^{(j)} + [(G^{(j)})^T G^{(j)}]^{-1} (G^{(j)})^T [Y - g(X; \beta^{(j)})],$$

- algorytm zatrzymujemy w kroku m , jeśli

$$\|\beta^{(m)} - \beta^{(m-1)}\| \leq \varepsilon \quad \text{lub} \quad m = M,$$

gdzie zgodnie z treścią zadania kładziemy

$$\varepsilon = 0.0001, \quad M = 100.$$

- przyjmujemy wówczas, że $\hat{\beta} = \beta^{(m)}$ oraz $\hat{\sigma}^2 = \hat{\sigma}^2(\hat{\beta})$.

W przypadku gdy macierz $(G^{(j)})^T G^{(j)}$ jest nieodwracalna, liczymy uogólnioną macierz odwrotną np. za pomocą funkcji *ginv()* z pakietu MASS.

Implementujemy zatem opisany wyżej algorytm.

Rozpoczynamy od wzorów na $G^{(j)}$ w funkcji $G.j()$.

```

M <- 100
epsilon <- 0.0001

G.j<-function(x,beta.1,beta.2,beta.3){
  g.1 <- x / (x + beta.2)
  g.2 <- ((-beta.1) * x) / (beta.2 + x)^2
  g.3 <- rep(1, length(x))
  return(cbind(g.1, g.2, g.3))
}

```

Następnie implementujemy wzory na ciąg iteracji $\beta^{(j)}$ w funkcji *beta.j()*.

```

beta.j<-function(x,y,beta.1,beta.2,beta.3){
  Y_hat <- g(x, beta.1, beta.2, beta.3)
  residała <- y - Y_hat

  G <- G.j(x, beta.1, beta.2, beta.3)

  beta_curr <- c(beta.1, beta.2, beta.3)

  delta <- MASS::ginv(t(G) %*% G) %*% t(G) %*% residała

  beta_next <- beta_curr + as.vector(delta)
  return(beta_next)
}

```

Pozostaje już zaimplementować sam algorytm Gaussa-Newtona w funkcji *Gauss.Newton()*.

```

Gauss.Newton<-function(x,y,beta.1,beta.2,beta.3,M,epsilon){

  beta_curr <- c(beta.1, beta.2, beta.3)
  n <- length(y)
  history <- list()

  for(m in 1:M){
    Y_hat <- g(x, beta_curr[1], beta_curr[2], beta_curr[3])
    sigma_sq <- sum((y - Y_hat)^2) / n
    history[[m]] <- c(beta_curr, sigma_sq)

    beta_next <- beta.j(x, y, beta_curr[1], beta_curr[2], beta_curr[3])

    if(sqrt(sum((beta_next - beta_curr)^2)) <= epsilon){
      Y_hat_next <- g(x, beta_next[1], beta_next[2], beta_next[3])
      sigma_sq_next <- sum((y - Y_hat_next)^2) / n
    }
  }
  return(history)
}

```

```

    history[[m + 1]] <- c(beta_next, sigma_sq_next)
    break
  }

  beta_curr <- beta_next
}

wynik <- do.call(rbind, history)
rownames(wynik) <- NULL
return(wynik)
}

```

oraz funkcję wiarygodności opisaną powyżej.

```

wiarogodnosc<-function(x,y,beta.1,beta.2,beta.3,sigma){
  n <- length(y)
  Y_hat <- g(x, beta.1, beta.2, beta.3)

  L <- -(n/2) * log(2 * pi) - (n/2) * log(sigma) - sum((y - Y_hat)^2) / (2 * sigma)
  return(L)
}

```

5 Zadanie 4

kroki algorytmu	$\hat{\beta}_1^{(s)}$	$\hat{\beta}_2^{(s)}$	$\hat{\beta}_3^{(s)}$	$\hat{\sigma}^2^{(s)}$	$L\left(\hat{\beta}^{(s)}, \hat{\sigma}^2^{(s)}\right)$
0	8.00000	2.00000	1.00000	1.6173782	-16.5934174
1	8.72025	0.01766	0.03557	7.0016935	-23.9201456
2	8.39791	0.33692	-0.28738	0.7071344	-12.4567126
11 (ostatni)	9.88997	0.91709	-0.43021	0.1291433	-3.9552210

Estymator Beta 1 zbiega do swojej rzeczywistej wartości, jaką jest 10, podczas gdy estymatory beta 2 i 3 znacząco odbiegają od swoich prawdziwych wartości (obydwa o różnią się od prawdziwego o ponad 2.)

6 Zadanie 5

```

beta_prawdziwe <- c(10, 3, 2)
beta_est <- tab.1[nrow(tab.1), 1:3]

porownanie <- rbind("Prawdziwe parametry" = beta_prawdziwe,
                    "Estymatory z GN" = beta_est)

```

```
colnames(porownanie) <- c("$\\beta_1$", "$\\beta_2$", "$\\beta_3$")

knitr::kable(porownanie,
             digits = 5,
             caption = "Porównanie prawdziwych parametrów z estymatorami",
             escape = FALSE)
```

Tabela 1: Porównanie prawdziwych parametrów z estymatorami

	β_1	β_2	β_3
Prawdziwe parametry	10.00000	3.00000	2.00000
Estymatory z GN	9.88997	0.91709	-0.43021

Estymator Beta 1 zbiega do swojej rzeczywistej wartości, jaką jest 10, podczas gdy estymatory beta 2 i 3 znacząco odbiegają od swoich prawdziwych wartości (obydwa o różnią się od prawdziwego o ponad 2.)

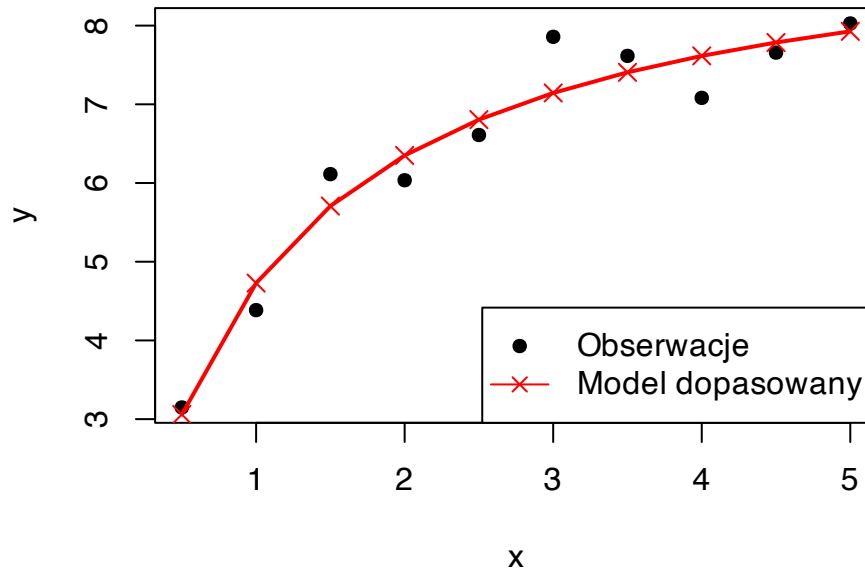
7 Zadanie 6

```
y_hat <- g(x, beta_est[1], beta_est[2], beta_est[3])

plot(x, y, pch = 16, col = "black",
     main = "Wartości oraz wartości dopasowane przez model",
     xlab = "x", ylab = "y")
points(x, y_hat, pch = 4, col = "red", cex = 1.2)
lines(x, y_hat, col = "red", lwd = 2)

legend("bottomright", legend = c("Obserwacje", "Model dopasowany"),
      col = c("black", "red"), pch = c(16, 4), lty = c(NA, 1))
```

Wartości oraz wartości dopasowane przez mode



Rysunek 1: Wykres porównujący obserwacje do dopasowanego modelu

Model stosunkowo dobrze dopasował się do wartości, zwłaszcza biorąc pod uwagę, że wartości do których model miał się dopasować nie mają prostej zależności liniowej.

8 Zadanie 7

Wobec wzoru na macierz kowariancji estymatora najmniejszych kwadratów w modelu regresji liniowej, przyjmujemy że (macierz G pełni rolę macierzy X z modelu regresji)

$$\text{Cov}(\hat{\beta}) = \sigma^2 (G^T G)^{-1}$$

Aby otrzymać estymator $\text{Cov}(\hat{\beta})$ zastępujemy nieznany wektor β (w G) wektorem $\hat{\beta}$, a wariancję σ^2 szacujemy za pomocą błędu średniokwadratowego

$$s^2 = \frac{\sum_{i=1}^n [y_i - g(x_i, \hat{\beta})]^2}{n - p}.$$

```
kowariancja <- function(s.kwadrat, G){  
  return(s.kwadrat * MASS::ginv(t(G) %*% G))  
}
```

```
n_obs <- length(y)  
p_parametry <- 3
```

```
beta_est <- tab.1[nrow(tab.1), 1:3]
```

```

y_hat_final <- g(x, beta_est[1], beta_est[2], beta_est[3])

s.kwadrat <- sum((y - y_hat_final)^2) / (n_obs - p_parametry)
G_est <- G.j(x, beta_est[1], beta_est[2], beta_est[3])

Cov_beta <- kowariancja(s.kwadrat, G_est)

nazwy_beta <- c("$\\beta_1$", "$\\beta_2$", "$\\beta_3$")
rownames(Cov_beta) <- nazwy_beta
colnames(Cov_beta) <- nazwy_beta

knitr::kable(Cov_beta,
              digits = 5,
              caption = "Oszacowana macierz kowariancji estymatora  $\\hat{\\beta}$ ",
              escape = FALSE)

```

Tabela 2: Oszacowana macierz kowariancji estymatora $\hat{\beta}$

	β_1	β_2	β_3
β_1	2.81464	-0.82230	-3.63704
β_2	-0.82230	0.33989	1.26593
β_3	-3.63704	1.26593	5.13321

Najmniejszą wariancję wykazuje beta2, z kolei beta3 ma bardzo wysoką. Można też zauważyć, że wartości kowariancji w żadnym przypadku nie są bliskie zeru, więc praktycznie każdy współczynnik jest skorelowany z pozostałą parą. Na wykresie wydawało się, że model jest dobry jednak ta tabela wykazuje jego niestabilność i zaraz w kolejnych zadaniach postaram się to pokazać.

9 Zadanie 8

Do samodzielnego zrobienia.

```

tab.2 <- Gauss.Newton(x, y, 9.7, 2.9, 2, M, epsilon)
tab.3 <- Gauss.Newton(x, y, 1, 1, 1, M, epsilon)

```

10 Zadanie 9

Do samodzielnego zrobienia.

```

zestawienie <- rbind(
  "Start: (8, 2, 1)" = tab.1[nrow(tab.1), ],
  "Start: (9.7, 2.9, 2)" = tab.2[nrow(tab.2), ],

```



```

      "Start: (1, 1, 1)"      = tab.3[nrow(tab.3), ]
    )

colnames(zestawienie) <- c("$\\beta_1$", "$\\beta_2$", "$\\beta_3$", "$\\sigma^2$")

knitr::kable(zestawienie,
             digits = 5,
             caption = "Porównanie estymatorów dla różnych punktów startowych",
             escape = FALSE)

```

Tabela 3: Porównanie estymatorów dla różnych punktów startowych

	β_1	β_2	β_3	σ^2
Start: (8, 2, 1)	9.88997	0.91709	-0.43021	0.12914
Start: (9.7, 2.9, 2)	9.88997	0.91709	-0.43022	0.12914
Start: (1, 1, 1)	0.41842	-1.17854	6.02286	1.73209

Po zmianie punktu startowego, w pierwszym przypadku nic się nie zmienia, natomiast w drugim podaje zupełnie inne oszacowania, natomiast są one kompletnie inne i znacząco różnią się od tych prawdziwych, więc algorytm G-N w tym przypadku nie spełnia dobrze zadania.